

Chapter 7. Foundation Models and Context Engineering

The number of Generative AI (GenAI) use cases has grown rapidly over the past few years, moving well beyond experimentation into applications that deliver tangible business value.

For example, [Airbnb completed a large-scale LLM-driven code migration](#) to update nearly 3,500 React component test files. It completed in just six weeks a project that had initially been estimated to require 1.5 years of manual engineering, by combining frontier models with robust automation.

Similarly, [DoorDash built AutoEval](#), an LLM-powered, human-in-the-loop system for evaluating the quality of search result pages. Instead of relying on slow and inconsistent human labeling, the system increased evaluation speed by 98% and expanded capacity by a factor of nine. Beyond scalability, it improved alignment with expert judgment and enabled continuous quality monitoring, allowing human experts to focus on higher-value tasks such as defining guidelines, handling edge cases, and improving models.

These examples illustrate a broader shift: organizations are no longer asking *if* they should use LLMs but *how* to operationalize them effectively. From my consulting experience, this is where the real challenge begins.

Introduction to LLMOps

Many teams are eager to build “agents” and experiment with increasingly complex LLM-powered systems, but far fewer have the infrastructure, processes, and practices required to run them reliably in production. The result is a growing gap between what is possible in demos and what can be sustained in real-world systems. This and the next two chapters focus on closing that gap. Together, they cover the full deployment lifecycle of LLM-powered systems much like we covered the deployment lifecycle of traditional ML systems in Chapters [2](#) through [6](#).

Rather than optimizing for the “best” possible model or architecture, the emphasis here is on *operational excellence*: how to build systems that are observable, maintainable, and scalable in practice. Topics such as preprocessing, chunking strategies, embedding selection, and architecture design are important, and we will reference them where relevant, but they are not the primary focus.

This field goes by several names, including LLMOps, AgentOps, and GenAIOps. We use LLMOps deliberately. Agents are our main examples, but they are one type of architecture among several, and many nonagen-tic LLM systems raise the same operational questions. As we discussed in [Chapter 1](#), what sets these systems apart from traditional ML is their non-determinism and their reliance on pre-trained foundation models. Naming the discipline after the model keeps it general and preserves the line from MLOps to LLMOps.

Before we go further, however, it is important to clarify a term that will appear frequently throughout this chapter: *agent*.

What Is an Agent?

The term *agent* is used extensively across the AI landscape, often with slightly different meanings, which can easily lead to confusion if left undefined.

In this book, we adopt a pragmatic definition inspired by the OpenAI Agent SDK:¹

An agent is a large language model (LLM) configured with instructions, a model, and access to tools.

This definition keeps things grounded in how systems are actually built and deployed. Concretely, an agent consists of three core components:

Instructions

These define the role and behavior of the model.

A model

This determines the agent’s underlying capabilities and parameters.

Tools

These extend the model's abilities through external functions or APIs.

Even within this setup, developers retain control over key aspects of execution, such as how many iterations are allowed, how conversation history is managed, which tools are exposed, and what parameters govern the model's behavior.

It is worth noting that not all organizations define agents in the same way. For example, in the article [“Building Effective Agents”](#), Anthropic distinguishes between an augmented LLM and an agent: an *augmented LLM* is simply a model with access to tools and memory, whereas an *agent* is a system that operates in a loop of *decide* → *act* → *observe*.

While this perspective is useful, especially when discussing more advanced systems, it introduces a boundary that is often less important in practice. In real-world applications, the more relevant question is not how we label the system but *who is making decisions*.

Here are two simple rules of thumb:

- If *your code* determines what actions are taken, the system is not an agent.
- If *the LLM* determines what actions are taken, the system is an agent.

This distinction helps clarify common patterns. A traditional retrieval-augmented generation (RAG) system, in which the application controls when documents are retrieved, is not an agent. However, once the LLM is responsible for deciding whether retrieval is necessary and which tools to call, the system becomes agentic.

This leads to an important conclusion: *not every system that uses an LLM is an agent*. From this perspective, agents are best understood not as a separate category but as a point on a spectrum. At one end are fully deterministic pipelines, in which every step is explicitly controlled. At the other are highly autonomous systems, in which the model dynamically decides how to achieve a goal. Most real-world systems fall somewhere in between.

In this chapter, the system we'll build sits at the simpler end of this spectrum. It is an agent in its most basic form: an LLM that is equipped with instructions and tools and is capable of making limited decisions. Despite its simplicity, it reflects the core patterns used in production systems.

Structure of the LLMOps Chapters

In this and the next few chapters, we will build a system that creates LinkedIn posts about recent arXiv papers on AI in a specific style. I realize this is a toy example and a different architecture might have made more sense for this use case, but my goal is to showcase all the Databricks components that are relevant for bringing AI systems to production.

Here is a list of the LLM system components we are going to build and discuss in the next few chapters:

Foundation models on Databricks (in [Chapter 7](#))

When you build these, you will be selecting and hosting the “brain” of your system.

Context engineering (in [Chapter 7](#))

This involves managing the external data and history that ground your agent, and it includes the following:

Getting from raw data to an agent context

You’ll do this to ingest and process external documents.

Using Model Context Protocol servers on Databricks

You’ll do this to standardize tool access.

Using agent memory with Lakebase

You’ll use this to maintain state across conversational sessions.

MLflow for AI agents (in [Chapter 8](#))

When you build this, you’ll be tailoring the experiment lifecycle for GenAI. The steps involved in this process are as follows:

Logging and registering an agent

This includes defining the logic of an agent, logging it in MLflow experiment tracking, and registering it in Unity Catalog.

MLflow tracing

This includes recording reasoning chains and tool calls for deep observability.

Agent evaluation

This involves systematically measuring output quality.

LLM system deployment (in [Chapter 9](#))

You'll do this to operationalize the workflow.

LLM system monitoring ([Chapter 9](#))

You'll do this to monitor safety and performance in the real world.

[Figure 7-1](#) provides a visual representation of these components.

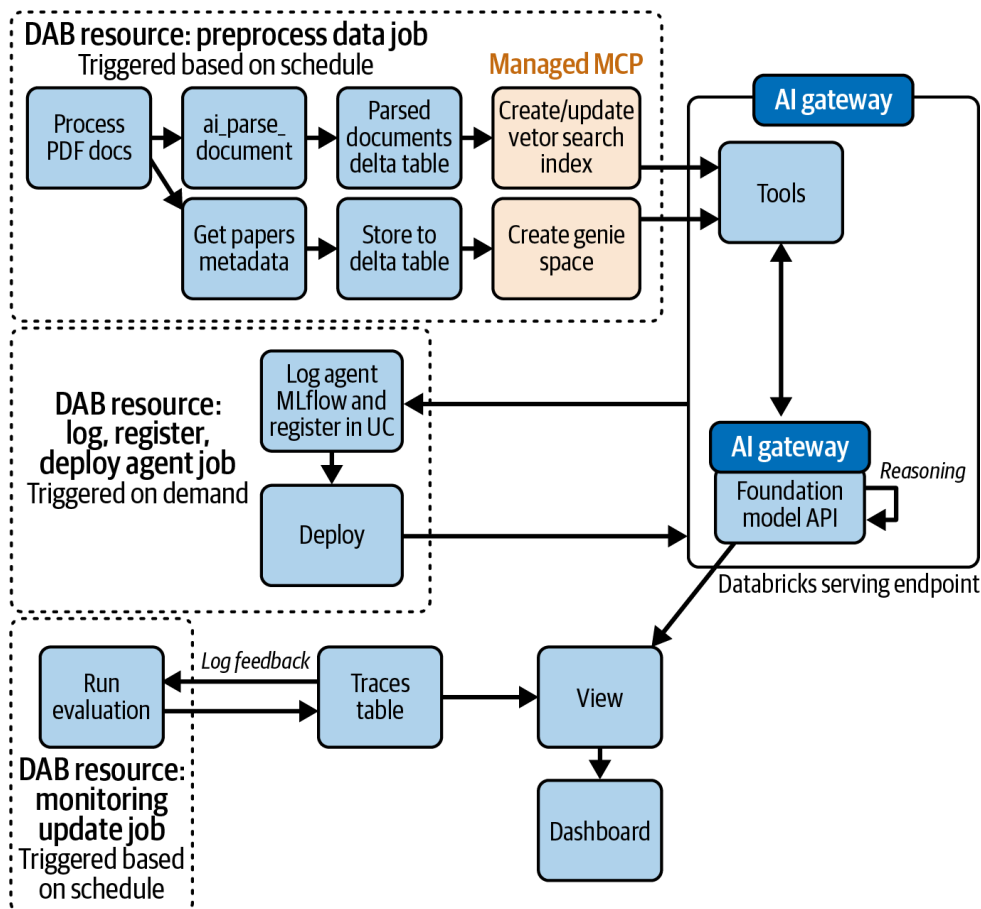


Figure 7-1. LLM system components

Let's go through these components one by one, starting with foundation models on the Databricks platform.

Foundation Models on Databricks

Working with foundation models on Databricks gives you flexibility: you can employ ready-to-use hosted models, deploy models with provisioned throughput, or connect to external providers like OpenAI or Anthropic. In

this section, we'll explore each option, discuss supported model types, and show you how to query them.

Hosting Options

The Databricks platform hosts a variety of ready-to-use foundation model APIs, including models like Meta's Llama 4, OpenAI's GPT-5.5, and Anthropic's Claude Opus 4.8. Beyond these hosted models, you can deploy open-source models through provisioned throughput serving endpoints, or you can connect to external providers like Azure OpenAI or Amazon Bedrock.

Regardless of which option you choose, all models use Databricks Model Serving, which follows the OpenAI API specification. It supports three endpoint types:

llm/v1/chat

This corresponds to the OpenAI Chat Completions API.

llm/v1/embeddings

This corresponds to the OpenAI Embeddings API.

llm/v1/completions

This follows the legacy OpenAI Completions API.

This unified interface lets you query foundation models in the same way, regardless of the provider or where the model is hosted. This consistency makes it much easier to experiment with different models, perform evaluations, and enable fallback strategies. You can even use an OpenAI client to interact with foundation models on Databricks. Note that at the time of writing, the responses API is only supported for Databricks-hosted OpenAI closed-sourced foundation models.

Here is an example of how we can interact with the `databricks-gpt-oss-120b` model:

```
from databricks.sdk import WorkspaceClient
from openai import OpenAI

w = WorkspaceClient()
host = w.config.host
token = w.tokens.create(lifetime_seconds=1200).token_value

client = OpenAI(
```

```

    api_key=token,
    base_url=f"{host}/serving-endpoints"
)

response = client.chat.completions.create(
    model="databricks-gpt-oss-120b",
    messages=[
        {"role": "user",
         "content": "What is MLOps?"}
    ]
)

```

Another benefit of using foundation models on Databricks is access to the Databricks Unity AI Gateway (see [Figure 7-2](#)). We deployed an AI gateway for our Model Serving endpoint earlier in the book and demonstrated how it enables usage tracking, payload logging, and traffic management. We can apply AI Gateway to any Databricks Model Serving endpoint, including foundation model APIs. We can also enable Databricks AI Guardrails to ensure PII data is deleted or masked and harmful content is blocked, for both model input and output.

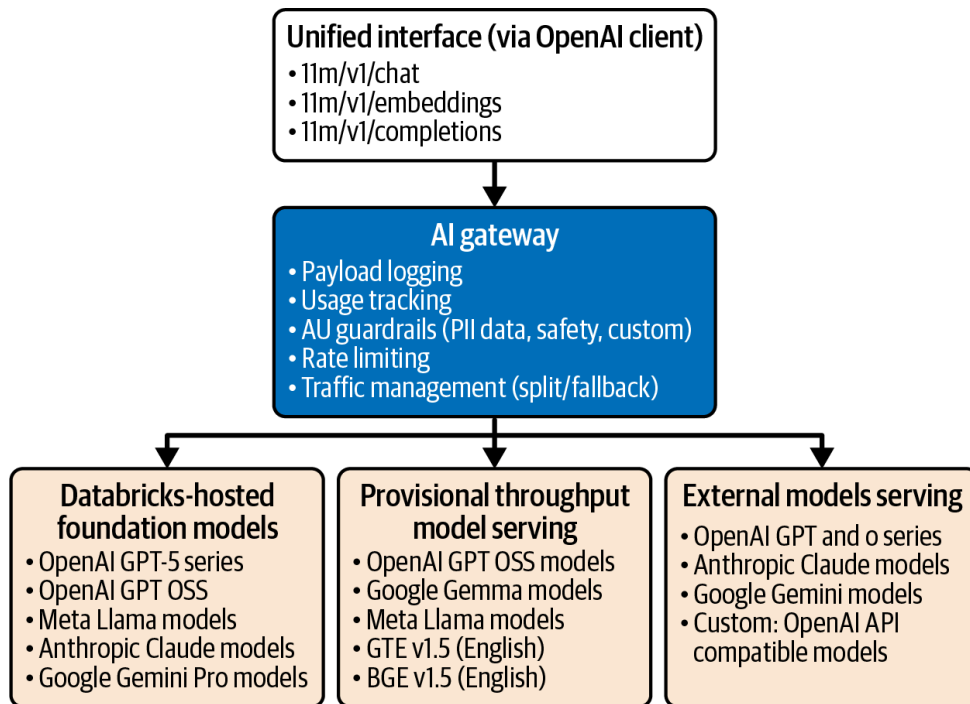


Figure 7-2. Depiction of Databricks AI Gateway

We can also use foundation model APIs in batch mode by using `ai_query`.² Some Databricks-hosted models are optimized for batch processing, but `ai_query` works for any endpoint deployed through Databricks Model Serving.

As we mentioned earlier, Databricks offers ready-to-use hosting options for foundation models. We discuss them in detail in the following sections.

Databricks-hosted models (pay-per-token)

These models are ideal for experimentation and exploration, as they require no additional configuration or upfront infrastructure commitment. The availability of these models in your workspace will depend on your cloud provider and region.³ However, they are subject to workspace-level rate limits in input/output tokens and queries per hour. The most restrictive rate limit applies at any given time; if you exceed it, you will receive a 429 error until the window resets.

Provisioned throughput endpoints

These endpoints are recommended for production, and they provide performance guarantees. You can deploy models behind provisioned throughput endpoints directly from the `system.ai` schema. You can also bring your own model (perhaps one you've fine-tuned) from sources like Hugging Face. If you use the latter approach, you'll need to log the model using MLflow (using the transformers flavor) and register it in Unity Catalog.

Provisioned throughput works only for selected foundation model families and for specific runtime configurations (size and context window) that Databricks has tuned and supports.⁴ If you choose to bring your own model and its metadata falls outside supported ranges, the provisioned throughput endpoint creation will fail too.⁵

For some models, provisioned throughput is expressed in tokens per second (input and output tokens combined), rather than requests per second. Since LLM requests vary widely in token length, requests per second is no longer a reliable measure of capacity. For example, if a request requires 2,048 input tokens and 256 output tokens (for 2,304 tokens in total), a provisioned throughput of 2,304 tokens per second will allow the endpoint to process one such request per second. If you increase this to 4,608 tokens per second, the endpoint will be able to process two such requests per second, with each request still taking roughly one second to complete.

Optimizing LLM inference performance is not an easy task, and GPU autoscaling is slower than CPU autoscaling for regular Model Serving endpoints. Databricks recommends conducting benchmarking to help you choose the correct range of tokens per second.⁶ You can also scale the

endpoint to 0 to avoid unnecessary costs, but that is not recommended for production environments.

For other models, provisioned throughput is expressed in **model units**, which is an abstraction layer on top of tokens per second. For the Llama 4 Maverick model, for example, 50 model units means approximately 3,250 tokens per second.

External models

External models are hosted outside of Databricks, which means you may need to configure firewall rules to access them.⁷ Various providers are supported, and you can also use custom providers as long as their APIs are OpenAI-compatible.

Even though you could access external models directly from the agent logic (as long as the secrets are configured and the networking is resolved), creating a serving endpoint that accesses an external model is a good idea because of the benefits mentioned previously, specifically around governance.

You can also use the custom provider option when you need to use functionalities, such as those offered by OpenAI, that are not yet covered by out-of-the-box solutions. For instance, while Databricks' existing OpenAI external Model Serving options currently support text and image inputs with text-only outputs, the Custom Provider solution enables support for images as output (encoded as base64) use cases through Databricks.⁸

Types of Foundation Models

Foundation models on Databricks are categorized into four functional types to help you select the most effective tool for your specific application:

General purpose models

These models are fine-tuned on large datasets of human dialogue, which enables them to generate contextually relevant responses in multiturn conversations. They are ideal for use cases such as customer support, virtual assistants, and interactive tutoring systems. Examples include Anthropic Claude models, Google Gemini models, and OpenAI GPT series models.⁹

Embedding models

These models transform complex data, such as text, images, and audio, into embeddings that capture the essential features and relationships within data. They are recommended for applications that require semantic understanding, similarity comparison, and efficient retrieval or clustering of complex data. Examples include Cohere, Databricks, OpenAI, and Google text embedding models.

Vision models

These models are designed to analyze visual data, and they are used for object recognition and image classification. Note that while they accept image inputs, standard configurations typically produce text outputs unless you're using a Custom Provider setup. Examples include Anthropic Claude models, Google Gemini models, and OpenAI GPT series models.

Reasoning models

These models excel at breaking down complex tasks and explaining their decision-making process. They are best suited to content creation, summarization, and agent orchestration. Examples include Anthropic Claude models, Google Gemini models, and OpenAI GPT series models with reasoning capabilities.

In this book, we are going to use the `databricks-gte-large-en` embedding model to generate embeddings and store them in the AI Search index. For the core LLM logic within the agent definition, we will use `databricks-gpt-oss-120b`.

Now that we have a better understanding of foundation models on Databricks, let's move on to one of the most important topics when it comes to building agents: context engineering.

Context Engineering

In the early days of AI engineering, it was all about prompt engineering. But while well-crafted system prompts remain important today, modern agent systems demand much more. We need strategies to manage the entire *context state*, which includes external data, available tools, conversation history, and intermediate reasoning steps (see [Figure 7-3](#)).

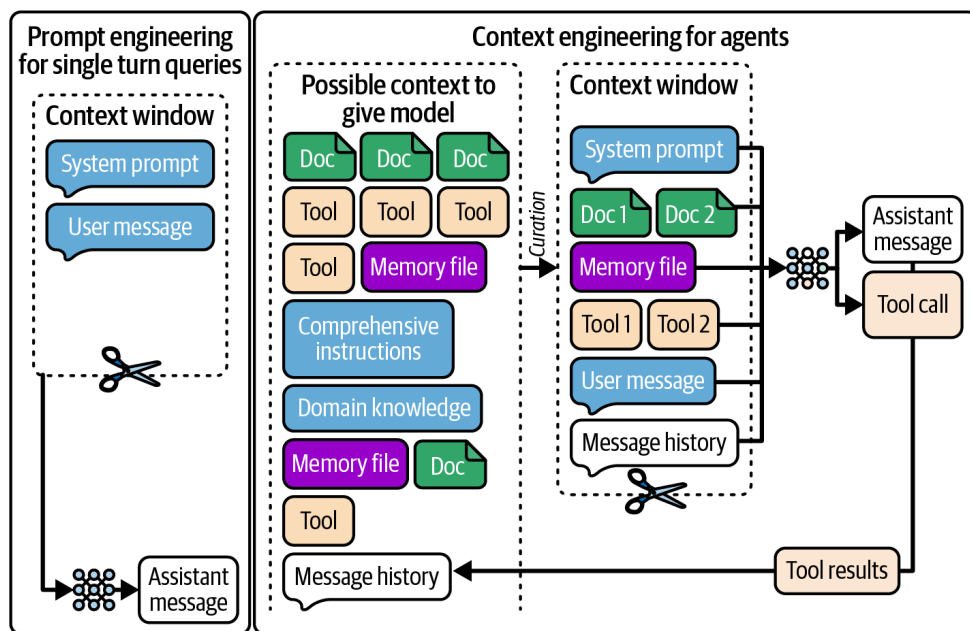


Figure 7-3. Prompt engineering vs context engineering, from [“Effective context engineering for AI agents”](#) by Anthropic

Consider an agent running in a loop. Each *turn* of the loop generates new information (tool outputs, retrieved documents, and user clarifications) that may be relevant for subsequent reasoning. But context windows are finite. The challenge isn’t just *getting* information to the model; it’s about shaping more behavior and *curating* what goes in, *when*, and *how*. This is what modern *context engineering* is all about.

In this section, we’ll explore three core aspects of context engineering:

Getting from raw data to agent context

This is how agents access and retrieve information beyond their training data.

Model Context Protocol servers on Databricks

This is a standardized protocol for exposing tools and data sources to agents.

Using agent memory with Lakebase

This is how you maintain state and conversation history across interactions.

Reference Architecture: The arXiv Curator

To make these concepts concrete, we’ll build a system that generates LinkedIn posts about recent arXiv papers on AI that are written in a consistent style. To achieve this, our system will perform the following steps:

1. Download and process full paper PDFs by extracting the complete text, not just metadata. This will give us access to results, examples, and insights buried deep in the paper.
2. Chunk and embed the content by storing vector representations in a Databricks AI Search index for semantic retrieval.
3. Track paper metadata in a Delta table by recording which papers we've processed, when, and their key attributes. Then, we'll also create a Genie agent (formerly, Genie space) on top of this table for natural language querying.
4. Expose these resources via Model Context Protocol (MCP). Both the AI Search index and Genie agent are available as managed Databricks MCP servers. This means our agent can access them through a standardized tool interface, which we'll explore in detail in the next section.

Now, we could take a simpler approach: the arXiv API already provides titles and abstracts, which might be sufficient for basic posts. But we want to create higher-quality, more insightful content that's hard to create based on the abstracts.

I intentionally made the architecture more sophisticated than is strictly necessary for our toy example. These patterns (PDF ingestion, vector retrieval, structured metadata, and tool protocols) appear in nearly every production LLM system. Building them here, even for a simple use case, teaches transferable skills you'll apply to more complex scenarios.

For a real production system, though, I encourage you to think the other way around: go simple first and more complex only if necessary. I recommend checking out the [blog by Anthropic on building effective agents](#) to learn about using different complexity levels when it comes to AI systems.

Now, let's dive into the details of how we can get from raw data to an agent context for our use case.

Getting from Raw Data to Chunks

Before we dive into the ingestion and data processing logic, let's review the project configuration, as it provides important context for the steps that follow.

Project configuration

The code for this LLMOps chapter is organized like the earlier MLOps examples. You can find it in the same [GitHub repository](#) under the *llmops* folder.

This is what the *project-config.yml* file looks like:

```
system_prompt: |
  You are a LinkedIn content creation assistant specialized
  in generating engaging posts about AI and machine learning research.

  Your role is to:
  1. Search for relevant arXiv papers using the AI Search tool
     based on the user's topic
  2. Query the Genie agent for additional context and insights
     about the research area
  3. Generate a LinkedIn post in a professional yet engaging style that:
     - Highlights key findings or innovations from recent research
     - Makes complex technical concepts accessible to a broad audience
     - Includes relevant paper citations and IDs
     - Uses a conversational tone appropriate for LinkedIn
     - Incorporates 2-3 relevant hashtags
     - Keeps the post concise (150-250 words)

  Always ground your posts in actual research findings from the tools
  available to you.

  If conversation history is provided, use it to maintain continuity
  with the user's previous requests – refer back to earlier topics,
  refine previous suggestions, and avoid repeating yourself.

experiment_path: /Shared/genai-arxiv-agent
usage_policy_id: 686c435d-90af-4c40-9ca4-442d4a3d8899
lakebase_project_id: arxiv-agent-lakebase

dev:
  catalog: mlops_dev
  schema: arxiv
  volume: arxiv_papers
  genie_space_id: 01f12f9b759a1da0807c0ae7366ee04d
  llm_endpoint: databricks-gpt-oss-120b
  warehouse_id: 1883724c3376103a

acc:
  catalog: mlops_acc
  schema: arxiv
  volume: arxiv_papers
  genie_space_id: 01f12f9b759a1da0807c0ae7366ee04d
```

```

    llm_endpoint: databricks-gpt-oss-120b
    warehouse_id: 1883724c3376103a

prd:
  catalog: mlops_prd
  schema: arxiv
  volume: arxiv_papers
  genie_space_id: 01f12f9b759a1da0807c0ae7366ee04d
  llm_endpoint: databricks-gpt-oss-120b
  warehouse_id: 1883724c3376103a

```

Notice the usage policy ID in the *project_config.yml* file. It differs from the one we used earlier. For the LLMOps chapter, we created a separate usage policy to isolate and track costs independently (see [Figure 7-4](#)). When we apply the LLMOps usage policy, it automatically adds the `project=arxiv_curator` tag, rather than `project=hotel_booking`, which we used previously.

Workspace settings > **Compute** >

Serverless usage policies ⓘ

Filter serverless usag... ☒ Usable in current workspace 2 results found Reset filters Create	
Name 	Policy ID
llmops_usage_policy	686c435d-90af-4c40-9ca4-442d4a3d8899
mlops_usage_policy	cdad6f29-a213-4525-9330-e25b90afbe7a

Figure 7-4. Serverless usage policies created for the MLOps and LLMOps chapters

You may notice that the policy again includes three environment-specific configurations, along with a shared configuration for the system prompt. We'll examine these in detail later in this chapter. Although we could use the MLflow Prompt Registry instead, it is primarily beneficial for prompt optimization workflows, and we would have to combine it with managing prompts as code.

Just as when we were processing data for our ML use case, we use Pydantic to verify the configuration, and define the `ProjectConfig` class under the `config` module:

```

import yaml
from pydantic import BaseModel

class ProjectConfig(BaseModel):
    """Configuration for the arxiv curator project."""

```

```

catalog: str
schema: str
volume: str
genie_space_id: str
llm_endpoint: str
system_prompt: str
warehouse_id: str
experiment_path: str
usage_policy_id: str | None = None
lakebase_project_id: str | None = None

@classmethod
def from_yaml(
    cls: "ProjectConfig", config_path: str, env: str = "dev"
) -> "ProjectConfig":
    """
    Load configuration from YAML file.

    Args:
        config_path: Path to the project_config.yml file
        env: Environment name (dev, acc, prd)

    Returns:
        ProjectConfig instance
    """
    if env not in ["prd", "acc", "dev"]:
        raise ValueError(
            f"Invalid environment: {env}. Expected 'prd', 'acc', or 'dev'"
        )

    with open(config_path) as f:
        config_dict = yaml.safe_load(f)
        env_config = config_dict[env]
        env_config["system_prompt"] = config_dict["system_prompt"]
        env_config["experiment_path"] = config_dict["experiment_path"]
        env_config["usage_policy_id"] = config_dict.get("usage_policy_id")
        env_config["lakebase_project_id"] = config_dict.get("lakebase_project_id")
        return cls(**env_config)

```

Now that we understand the project configuration, we can use it in the data ingestion logic.

Data ingestion

We will utilize the [arXiv Python package](#) to search for the latest papers in the AI category from the ArXiv database. We will then download the papers, store them in a volume, and save all the metadata in the

`arxiv_papers` table within the configured catalog and schema. The code shown here can be found in the GitHub repository under *notebooks/chapter7_1.arxiv_process_docs.py* in the *llmops* folder. The packaged version of the code can be found in the *src/arxiv_curator* folder under the `data_processor` module.

Here, we use incremental processing based on timestamps. If the `arxiv_papers` table exists, the system queries the timestamp of the most recently processed paper and searches arXiv for all papers published between that timestamp and the current execution time. This ensures that only new papers will be downloaded and processed, to avoid redundant work.

NOTE

Databricks Free Edition has network restrictions that block outbound API calls to certain external services. If you use the Free Edition, you won't be able to call the arXiv API in a Databricks notebook. Instead, you should consider developing locally and then uploading files to Unity Catalog Volumes using Databricks CLI.

First, let's load the project configuration and retrieve the catalog name, schema name, and volume name:

```
from loguru import logger
from pyspark.sql import SparkSession

from arxiv_curator.config import ProjectConfig

spark = SparkSession.builder.getOrCreate()

cfg = ProjectConfig.from_yaml("../project_config.yml")
catalog = cfg.catalog
schema = cfg.schema
volume = cfg.volume
```

Next, we need to make sure the schema and volume exist. (Note that we're reusing the same catalog that we used in the MLOps chapters.):

```
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.{schema}")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {catalog}.{schema}.{volume}")
```

For initial runs in which the `arxiv_papers` table does not yet exist, the system retrieves papers from the last three days. This provides a mean-

ingful initial dataset without overwhelming the system with arXiv's extensive historical archives:

```
import time

import arxiv

client = arxiv.Client()
metadata_table = f"{catalog}.{schema}.arxiv_papers"

if spark.catalog.tableExists(metadata_table):
    start = str(
        spark.sql(f"""
            SELECT max(processed)
            FROM {metadata_table}
            """).collect()[0][0]
    )
else:
    start = time.strftime("%Y%m%d%H%M", (time.gmtime(time.time() - 24 * 3600 * 3))

end = time.strftime("%Y%m%d%H%M", time.gmtime())

search = arxiv.Search(query=f"cat:cs.AI AND submittedDate:[{start} TO {end}]")
papers = client.results(search)
```

Once the system retrieves the search results, it processes each paper individually. For every paper, it extracts the following metadata: paper ID, title, author list, abstract, URL, and publication timestamp. Additionally, the system appends processing metadata, including the timestamp when the paper was processed and the volume path where the downloaded paper is stored:¹⁰

```
import os

records = []
pdf_dir = f"/Volumes/{catalog}/{schema}/{volume}/{end}"
os.makedirs(pdf_dir, exist_ok=True)

for paper in papers:
    paper_id = paper.get_short_id()
    # download PDF
    try:
        paper.download_pdf(dirpath=pdf_dir, filename=f"{paper_id}.pdf")
        # collect metadata (keep datetime intact)
        records.append(
            {
                "paper_id": paper_id,
```

```

        "title": paper.title,
        "authors": [author.name for author in paper.authors],
        "summary": paper.summary,
        "pdf_url": paper.pdf_url,
        "published": int(paper.published.strftime("%Y%m%d%H%M")),
        "processed": int(f"{end}"),
        "volume_path": f"{pdf_dir}/{paper_id}.pdf",
    }
)
except Exception:
    logger.warning(f"Paper {paper_id} was not successfully processed.")
    pass
time.sleep(3)  # to avoid rate limiting

```

Finally, we write the retrieved metadata into the `arxiv_papers` table:

```

from pyspark.sql import functions as F
from pyspark.sql import types as T

if len(records) > 0:
    metadata_schema = T.StructType(
        [
            T.StructField("paper_id", T.StringType(), False),
            T.StructField("title", T.StringType(), True),
            T.StructField("authors", T.ArrayType(T.StringType()), True),
            T.StructField("summary", T.StringType(), True),
            T.StructField("pdf_url", T.StringType(), True),
            T.StructField("published", T.LongType(), True),
            T.StructField("processed", T.LongType(), True),
            T.StructField("volume_path", T.StringType(), True),
        ]
    )

    metadata_df = spark.createDataFrame(records, schema=metadata_schema).withColumn(
        "ingest_ts", F.current_timestamp()
    )
    metadata_df.write.format("delta").mode("append").saveAsTable(f"{metadata_table}")

```

Now that the papers are stored in the volume and all the metadata is stored in a Delta table, we're going to tackle PDF processing—one of those problems that looks simple until you actually try to do it at scale.

PDF processing

PDFs are everywhere in enterprise environments, and they're consistently one of the hardest data sources to work with reliably. That's be-

cause PDFs weren't designed for data extraction. They were designed to ensure that a document looks exactly the same all the time, no matter where you view or print it.

PDFs are built for visual presentation, but they lack metadata to process content reliably. They may look organized on screen, but they are unstructured behind the scenes. This makes them incredibly frustrating to work with when you need to extract structured data for processing, searches, or feeding into your AI systems.

You may find several options when it comes to PDF processing (including scanned documents): open-source projects like [Docling](#), or perhaps solutions provided by cloud vendors like Amazon Textract and Azure Document Intelligence. Also, modern LLMs with vision capabilities can now process PDFs directly. Models like GPT-5, Claude Sonnet 4.5, and Gemini 3 Pro can see the document layout, understand context, handle both native text and scanned images, and extract structured data in a single pass.

Databricks also provides an `ai_parse_document` function that significantly simplifies the PDF processing pipeline and handles all of this complexity behind the scenes, providing high accuracy for a [competitive price](#). Let's see how it works in practice.

NOTE

The `ai_parse_document` function is not supported while executing code locally via VS Code.

First, let's create a table called `ai_parsed_docs` to store our extraction results. We'll insert the document volume path alongside the parsed content, which comes from the `ai_parse_document` function running against all the PDF files we downloaded earlier in this pipeline:

```
spark.sql(
    f"""
    CREATE TABLE IF NOT EXISTS
    {catalog}.{schema}.ai_parsed_docs (
    path STRING,
    parsed_content STRING,
    processed LONG)
    """
)
```

```

spark.sql(

    f"""
INSERT INTO {catalog}.{schema}.ai_parsed_docs
SELECT
path,
ai_parse_document(content) AS parsed_content,
{end} AS processed
FROM READ_FILES(
    "{pdf_dir}",
    format => 'binaryFile')
    """
)

```

The output of the `ai_parse_document` function consists of elements (chunks) of the following types:

caption

This includes image and figure captions.

figure

This includes figures and images.

footnote

This includes footnotes.

page_header

This includes page headers (like page numbers).

section_header

This includes section headings.

table

This includes tables.

text

This includes regular text content.

title

This includes the document title.

[Figure 7-5](#) shows this output.

The `ai_parse_document()` function returns a document that's already broken into pieces along its own structure, with each piece tagged by type. For retrieval, the type that matters most is `text`, since that is where the answerable content sits. In effect, the parser has made the first chunking decision for us, one driven by the document's structure.

In this chapter, we go with the chunking that `ai_parse_document()` produces, to keep things simple. However, it is still worth it to learn about chunking strategy in its own right, since the parser's structural split is not always the one that serves retrieval best.

Sample	 path	 parsed_content
56	 dbfs:/Volumes/mlops_dev/arxiv/arxiv_papers/202511191940/2511.14715v1.pdf	 {"document":{"elements":[{"bbox":{"coord": [931,49,938,60],"page_id":0},"content":"1","description":null,"id":0,"type":"page_header"}, {"bbox":{"coord": [82,102,938,183],"page_id":0},"content":"FLARE: Adaptive Multi-Dimensional Reputation for Robust Client Reliability in Federated Learning","description":null,"id":1,"type":"title"}, {"bbox":{"coord": [82,199,937,240],"page_id":0},"content":"Abolfazi Younesi* (Student, IEEE), Leon Kiss* Zahra Najafabadi Samani, Juan Aznar Poveda*, Thomas Fafinger* (Member, IEEE)","description":null,"id":2,"type":"text"}, {"bbox":{"coord": [82,315,509,828],"page_id":0},"content":"Abstract—Federated learning (FL) enables collaborative model training while preserving data privacy. However, it remains vulnerable to malicious clients who compromise model integrity through Byzantine attacks, data poisoning, or adaptive adversarial behaviors. Existing defense mechanisms rely on static thresholds and binary classification, failing to adapt to evolving client behaviors in real-world deployments. We propose FLARE, an adaptive reputation-based framework that transforms client reliability assessment from binary decisions to a continuous, multi-dimensional trust evaluation. FLARE integrates: (i) a multi-dimensional reputation score capturing performance consistency, statistical anomaly indicators, and temporal behavior, (ii) a self-calibrating adaptive threshold mechanism that adjusts security strictness based on model convergence and recent attack intensity, (iii) reputation-weighted aggregation with soft exclusion to proportionally limit suspicious contributions rather than eliminating clients outright, and (iv) a Local Differential Privacy (LDP) mechanism enabling reputation scoring on privatized client updates. We further introduce a highly evasive Statistical Mimicry (SM) attack, a benchmark adversary that blends honest gradients with synthetic perturbations and persistent drift to remain undetected by traditional filters. Extensive experiments with 100 clients on MNIST, CIFAR-10, and SVHN demonstrate that FLARE maintains high model accuracy and converges faster than state-of-the-art Byzantine-robust methods under diverse attack types, including label flipping, gradient scaling, adaptive attacks, ALIE, and SM. FLARE improves robustness by up to 16% and preserves model convergence within 30% of the non-attacked baseline, while achieving strong malicious-client detection performance with minimal computational overhead. (https://github.com/Anonymous0-paper/FLARE)","description":null,"id":3,"type":"text"}, {"bbox":{"coord": [519,315,947,868],"page_id":0},"content":"aggregates them into a global model [5]. This decentralized approach not only minimizes data exposure but also reduces the burden of data transfer and storage, making FL particularly suitable for large-scale, privacy-sensitive applications. The promise of FL lies in its ability to harness the collective intelligence of numerous devices while respecting individual privacy [6]–[8]. Despite its advantages, the distributed and open nature of FL introduces security and trust challenges. Unlike traditional centralized training, where the data and computation are under the control of a single trusted entity, FL operates in a loosely coordinated environment where clients are autonomous and often unverified [9], [10]. This autonomy creates opportunities for malicious or unreliable participants to inject harmful updates, skew the global model, or exploit the system for personal gain. The lack of direct oversight, combined with the diversity of hardware, network conditions, and user behaviors, further complicates the task of ensuring consistent and trustworthy collaboration. As an illustrative example, Figure 1 demonstrates how traditional static approaches to client reliability assessment can suffer from critical misclassification errors: they may permanently exclude honest clients experiencing transient network issues or possessing unique data distributions, while failing to detect sophisticated adaptive attackers who manipulate their behavior to evade detection. As FL deployments scale across thousands of geographically dispersed devices with varying trustworthiness, the need for robust, adaptive mechanisms to assess and manage client

Figure 7-5. Output of `ai_parse_document` function

Chunking

Chunking decides what the retriever can find. A *chunk* is the unit that gets embedded and later returned, so if the answer to a question is split across two chunks or buried in a chunk full of unrelated text, then retrieval suffers no matter how strong the rest of the pipeline is. The strategies we cover in the following sections make trade-offs in three areas: how well a chunk respects the content's structure, how much it inflates your index, and how it fits inside your embedding model's limits.

Chunking strategies

The simplest approach splits documents into equal-length chunks by some unit: characters, words, sentences, or paragraphs. It works well enough for many use cases but has one fundamental problem: it ignores the structure of your content. A 2,048-character chunk may cut off mid-sentence, mid-paragraph, or in the middle of a critical explanation, and the context at those boundaries may therefore be lost.

Overlap is the usual fix. With 2,048-character chunks, you can carry 200 characters of overlap between them, so information near a boundary will still land intact in at least one chunk. The cost is duplication: you will store some content twice, and your index will grow.

A recursive split respects structure more directly. Take an arXiv paper: start at the section boundaries (Abstract, Introduction, Methods, and Results), and if a section exceeds your maximum chunk size, divide it further by subsection (Data Collection, Model Architecture, and Training Procedure), and finally by paragraph. This hierarchy keeps related content together and avoids breaking an explanation across an arbitrary boundary, so what comes back stays contextually complete.

Embedding window considerations

Chunk size is not entirely your choice. It is bounded by the embedding window of the model you pick. The Databricks GTE model has a window of 8,192 tokens, while Databricks BGE Large has 512. If a chunk exceeds the window, it cannot be embedded in full and part of what you intended to index will never make it in.

These limits are measured in tokens, not characters, and the two are not interchangeable. A *token* can be a word, a subword, or a punctuation mark, depending on the model's tokenizer. The phrase “MLOps is awesome!” consists of 17 characters but four tokens: "MLOps", "is", "awesome", and "!" . When you set a chunk size in characters, keep this conversion in mind so your chunks stay inside the window.

Processing chunks

To make the data usable for vector search, we need to create another table that stores each chunk in a separate row. Let's create a user-defined function to extract the chunks. Here, we are only extracting chunks of type `text`:

```
import json
import re

from pyspark.sql.functions import (
    col,
    concat_ws,
    explode,
    udf)

from pyspark.sql.types import (
```

```

ArrayType,
StringType,
StructField,
StructType)

def extract_chunks(parsed_content_json: str) -> list[tuple[str, str]]:
    parsed_dict = json.loads(parsed_content_json)
    chunks = []

    for element in parsed_dict.get("document", {}).get("elements", []):
        if element.get("type") == "text":
            chunk_id = element.get("id", "")
            content = element.get("content", "")
            chunks.append((chunk_id, content))
    return chunks

chunk_schema = ArrayType(
    StructType(
        [
            StructField("chunk_id", StringType(), True),
            StructField("content", StringType(), True),
        ]
    )
)

extract_chunks_udf = udf(extract_chunks, chunk_schema)

```

We also want to extract the paper ID from the path and clean chunks (remove whitespaces, fix hyphenation, and collapse new lines into spaces):

```

def extract_paper_id(path):
    return path.replace(".pdf", "").split("/")[-1]

extract_paper_id_udf = udf(extract_paper_id, StringType())

def clean_chunk(text: str) -> str:
    # fix hyphenation across line breaks:
    # "docu-\nments" => "documents"
    t = re.sub(r"(\w)-\s*\n\s*(\w)", r"\1\2", text)

    # collapse internal newlines into spaces
    t = re.sub(r"\s*\n\s*", " ", t)

    # collapse repeated whitespace
    t = re.sub(r"\s+", " ", t)

    return t.strip()

```

```
clean_chunk_udf = udf(clean_chunk, StringType())
```

Finally, we can create the `arxiv_chunks` table and enable

`ChangeDataFeed` so that we can use the table to create and incrementally update the AI Search index. Notice that we are joining some fields from the metadata table so that we can perform reranking and filtering later:

```
df = spark.table(f"{catalog}.{schema}.ai_parsed_docs").where(f"processed = {end}")

# Load metadata table
metadata_df = spark.table(metadata_table).select(
    col("paper_id"),
    col("title"),
    col("summary"),
    concat_ws(", ", col("authors")).alias("authors"),
    (col("published") / 100000000).cast("int").alias("year"),
    ((col("published") % 100000000) / 1000000).cast("int").alias("month"),
    ((col("published") % 1000000) / 10000).cast("int").alias("day"),
)

# Create the transformed table
chunks_df = (
    df.withColumn("paper_id", extract_paper_id_udf(col("path")))
    .withColumn("chunks", extract_chunks_udf(col("parsed_content")))
    .withColumn("chunk", explode(col("chunks")))
    .select(
        col("paper_id"),
        col("chunk.chunk_id").alias("chunk_id"),
        clean_chunk_udf(col("chunk.content")).alias("text"),
        concat_ws("_", col("paper_id"), col("chunk.chunk_id")).alias("id"),
    )
    .join(metadata_df, "paper_id", "left")

# Write to table
chunks_table = f"{catalog}.{schema}.arxiv_chunks"
chunks_df.write.mode("append").saveAsTable(chunks_table)

spark.sql(
    f"""ALTER TABLE {chunks_table}
    SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
    """
)
```

At this point in the pipeline, we have the arXiv papers stored as PDFs in volumes, their metadata stored in Delta tables, the parsed content ex-

tracted with `ai_parse_document()`, and the chunks stored in a Delta table. What we still lack is a way to find the right chunks for a given question. That is what Databricks AI Search provides, and setting it up is the focus of the next section.

Databricks AI Search Index

Vector search is fundamentally different from traditional search. Instead of matching keywords, it works with *embeddings*—mathematical representations (vectors) of semantic content. Text with similar meanings produces similar vectors, even if the exact words are different.

For example, in scientific papers, researchers don't always use the same terminology. One paper may discuss convolutional neural networks while another talks about CNNs or ConvNets. Databricks AI Search handles this naturally because it understands that these concepts are semantically related.

Creating the endpoint and index

To get started, we must first create an AI Search endpoint. Databricks AI Search provides standard and storage-optimized endpoint configurations. Standard endpoints have a capacity of 320 million vectors at dimension 768, while storage-optimized ones can accommodate over one billion vectors of that dimension and provide 10 to 20 times faster indexing. Queries on storage-optimized endpoints have a slightly increased latency, and there are some differences in supported operations among different endpoint types.¹¹

We can create an AI Search endpoint programmatically by using the `VectorSearchClient`. Notice that here, we pass the usage policy ID that we will use later for cost allocation related to the AI Search endpoint:

```
from databricks.vector_search.client import VectorSearchClient

from arxiv_curator.config import ProjectConfig

cfg = ProjectConfig.from_yaml("../project_config.yml")
catalog = cfg.catalog
schema = cfg.schema

vsc = VectorSearchClient()
vector_search_endpoint_name = "vector-search-arxiv-endpoint"
if not vsc.endpoint_exists(name=vector_search_endpoint_name):
    vsc.create_endpoint_and_wait()
```

```
name=vector_search_endpoint_name,  
endpoint_type="STANDARD",  
usage_policy_id=cfg.usage_policy_id)
```

Once the endpoint is active, we can proceed with creating an AI Search index. We use the `create_delta_sync_index` function from the `VectorSearchClient`. This function takes the following arguments:¹²

columns_to_sync

These are the columns to transfer from the source table. If this argument is undefined, all columns will be synced.

endpoint_name

This is the name of the AI Search endpoint where the index is served.

source_table_name

This is the Delta table that contains the input data.

index_name

This is the full name for the new index.

primary_key

This is the unique identifier (in our case, the ID from the chunks table).

embedding_source_column

This is the column that contains the text to be embedded.

embedding_model_endpoint_name

This is the foundation model API that is used to generate embeddings. (We use the Databricks-hosted databricks-gte-large-en model.)

pipeline_type

This is the type of Lakeflow SDP that is used for synchronization. (We have chosen “TRIGGERED”.)

Notice that here, we pass the usage policy ID that we will use later for cost allocation related to the AI Search index. (So think about costs related to Lakeflow SDP runs.):

```

vs_index_fullname = f"{catalog}.{schema}.arxiv_index"
embedding_model_endpoint = "databricks-gte-large-en"

if not vsc.index_exists(
    endpoint_name=vector_search_endpoint_name,
    index_name=vs_index_fullname,
):
    index = vsc.create_delta_sync_index(
        endpoint_name=vector_search_endpoint_name,
        source_table_name=f"{catalog}.{schema}.arxiv_chunks",
        index_name=vs_index_fullname,
        pipeline_type="TRIGGERED",
        primary_key="id",
        embedding_source_column="text",
        embedding_model_endpoint_name=embedding_model_endpoint,
        usage_policy_id=cfg.usage_policy_id
    )
else:
    index = vsc.get_index(index_name=vs_index_fullname)
    index.sync()

```

When the new data is available in the source table, we will have to run the sync command:

```

index = vsc.get_index(index_name=vs_index_fullname)
index.sync()

```

As we discussed in earlier chapters regarding Feature Serving and the Databricks Online Feature Store, the same logic applies to synchronization modes here: we use `TRIGGERED` mode unless we consistently need fresh data. For lower latency requirements, `CONTINUOUS` mode ensures that updates arrive within seconds.

In the Catalog Explorer, you can view which Lakeflow SDP is associated with the AI Search index, along with its current status (see [Figure 7-6](#)).

arxiv_index

OverviewColumnsSample DataDetailsPermissionsHistoryLineageInsightsQuality

Overview

Index status

Online

Index type

Delta Sync

Source table

mlops_dev.arxiv.arxiv_chunks

Serving endpoint

vector-search-arxiv-endpoint

Sync schedule

Triggered

Rows indexed

27,392

MCP server URL

https:///api/2.0/mcp/ai-search/mlops_dev/arxiv/arxiv_index

Search Quality Evaluation

Beta

Send feedback

No evaluations yet

Evaluate search quality

Budget Policy

Budget policy changes will take effect on the next index sync.

llmops_usage_policy

arxiv_curator

Data Ingest

Pipeline id

6be5d2ab-99fe-422f-8f2a-1e80e9598459

Update status

Completed

Last processed commit

1

Figure 7-6. Databricks AI Search index view in the Catalog Explorer UI

It's important to mention here that the `create_delta_sync_index()` command does not just create the AI Search Index but also computes the embeddings. This allows us to query the index using natural language. The same model we use to create the embeddings is automatically applied to our query to retrieve the most relevant documents.

You can also bring your own embeddings. In this case, the source Delta table should contain precomputed embeddings instead of text. While this offers more control, it also means you will be responsible for generating the corresponding embeddings within your LLM application at query time. [Figure 7-7](#) demonstrates the differences between Databricks-managed and self-managed embedding workflows.

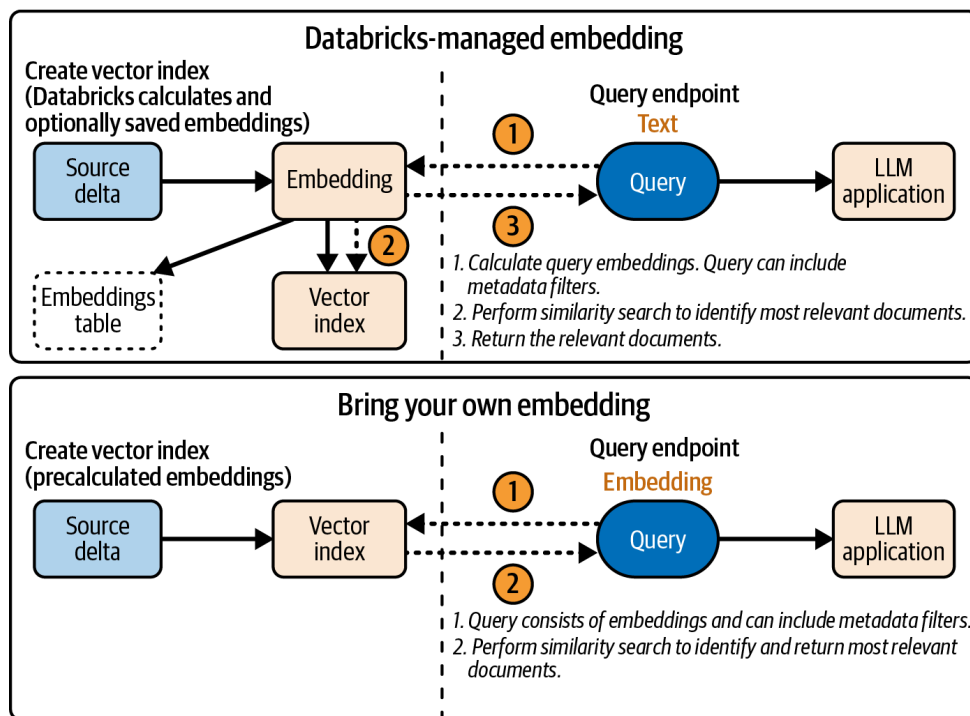


Figure 7-7. Databricks-managed versus self-managed embeddings, adapted from [the Databricks documentation](#)

Querying and retrieval quality

Databricks AI Search supports natural language queries. For the `similarity_search()` function, we need to provide the following arguments:

columns

These are the fields to return in the results.

query_text or query_vector

Here, we designate whether we want to make queries as a text or as embedding if we choose the “bring your own embedding” option.

filters

These are SQL-like filters that restrict results by metadata (e.g., specific years or authors). Filter operators like `=`, `>=`, `<=`, `IN`, `LIKE`, `NOT`, and `OR` are supported. Syntax varies by endpoint type.¹³

num_results

This is the number of chunks to retrieve.

score_threshold

This is where we define a minimum similarity score for results. Any document with a score below this threshold will be excluded,

even if it ranks within the top K results.

query_type (ANN or HYBRID)

Databricks AI Search uses the HNSW algorithm for its approximate nearest neighbor (ANN) searches and the L2 distance metric to measure embedding vector similarity.¹⁴ The hybrid keyword-similarity option matches exact words in the query while also using a vector-based similarity search to capture the semantic relationships and context of the query. This method is particularly useful in RAG applications where source data has unique keywords that are not well suited to pure similarity search.

reranker

Reranking is a technique that improves retrieval quality by evaluating the retrieved documents to identify the ones that are semantically most relevant. Databricks has developed a reranker that you can use right away, and you can also use it to specify columns containing additional context that is relevant for reranking, such as `paper.summary` and `title`.

The function also accepts operational arguments that control its runtime behavior: how verbose its logging is (the debugging level), how many times it retries on failure, the backoff factor between those retries, and whether warnings will be suppressed.

This is how we can use the `similarity_search()` function:

```
from databricks.vector_search.reranker import DatabricksReranker

results = index.similarity_search(
    query_text="Chunking strategies for document processing",
    columns=["text", "id"],
    filters={"year": "2026"},
    num_results=5,
    query_type = "hybrid",
    reranker=DatabricksReranker(columns_to_rerank=["text", "title", "summary"])
)
```

Optimizing search quality

To help you understand the impact of configuration changes, we need to establish evaluation metrics and a test dataset.

For a test dataset, you can use existing labeled query-answer pairs, generate synthetic test cases from your documents, or use LLM judges to assess quality without labels. Even a small synthetic dataset of 50 queries is enough to tell you whether hybrid search improves quality or whether reranking helps.

When it comes to evaluation metrics, you should pick them based on what failure looks like for your application. Once you have metrics aligned with your use case, you can systematically test configurations and determine which changes actually improve retrieval quality.

Key metrics include the following:

Recall@K

This measures how many relevant items were successfully retrieved. It's critical for legal and financial use cases.

Precision@K

This measures what percentage of retrieved items are actually relevant. It's critical for technical support where only the best result matters.

F1-score@K

This balances precision and recall.

The Databricks guide on vector search retrieval quality outlines eight main steps AI engineers should consider to improve quality:¹⁵

1. *Enable hybrid search.* One-line code change may improve recall by catching both semantic and keyword matches and improve precision for queries with specific terms.
2. *Use metadata filtering.* This reduces the search space, improves recall within the filtered subset, and improves precision by eliminating irrelevant results.
3. *Add reranking.* Databricks' built-in reranker is a great solution, but it is not suitable for high-QPS applications (those with > 5 QPS without additional scaling) or applications with low latency requirements. In those scenarios, it may be a good idea to build a custom reranker that utilizes a Model Serving endpoint.
4. *Improve data preparation.* Refine document parsing and chunking strategies.
5. *Optimize queries.* Generate multiple query variations to improve recall.

6. *Optimize prompts.* Use automatic prompt optimization techniques to improve prompts that are used for data preparation or query rewriting.
7. *Implement adaptive retrieval strategies.* Build agents that can reformulate queries or decide if retrieval is necessary.
8. *Fine-tune embedding models.* This is a final option if standard models do not provide satisfactory results, particularly for non-English languages.

So far, we've created an AI Search endpoint, built an AI Search index, and tested querying it. While vector search is perfect for retrieving relevant context based on similarity, our agent also needs tools for analytical tasks, such as aggregations and statistical queries. For those requirements, Databricks Genie is a great solution.

Genie Agent (Formerly, Genie Space)

Genie is a Databricks tool that lets business users get data insights by asking questions in natural language. It translates a question into a SQL query, runs it on a SQL warehouse, and returns an answer. Rather than relying on a single large model, Genie uses several specialized components to handle the steps from question to result.

How well Genie performs depends almost entirely on the context you give it. It draws on table schemas, column names and their descriptions, example SQL queries, custom instructions, and conversation history.

When that context is thin, or when Genie is pointed at large, raw, uncured tables, accuracy suffers. This is because the query space is wide, column meanings are ambiguous, and the model has more room to generate the wrong query. The fix is to design that context deliberately by doing the following:

Curate the datasets you expose

Genie chooses which tables and columns to use for each question, so a wide surface means more chances to join the wrong tables or pick a misleading column. Keep it scoped to the tables that actually answer your users' questions.

Aggregate upfront

Against raw data, Genie has to reconstruct the grouping, time windows, and deduplication for every question, and small variations

produce different numbers. Summary tables encode that logic once, so answers stay consistent.

Guide Genie with instructions and example SQL queries

Examples show Genie the join paths, filters, and column choices you treat as correct, so Genie will reuse them instead of improvising. Instructions cover what an example cannot be, such as which records to exclude or what specific metrics mean in your business.

Genie was originally built for people working through a UI, but it is now generally available through the Conversation API, which makes it usable inside AI agents and applications.

Databricks AI Search versus Genie

Databricks AI Search and Genie serve complementary purposes. To determine which one would be the best fit for a specific task, consider how the nature of the query dictates the tool usage by asking yourself these questions:

What are recent approaches to efficient attention mechanisms?

Databricks AI Search is the best fit here, as it retrieves semantically similar paper chunks based on meaning rather than structured counts.

How many papers were published in each category in 2026?

Genie is ideal for this as it translates this to SQL against the metadata table.

Who are the most prolific authors in our collection?

Genie can easily count and rank authors by querying the structured metadata.

For the arXiv pipeline, we will create a Genie agent and grant it access to the metadata table. Although it is possible to connect Genie directly to an AI Search index, keeping these components separate gives us finer control over how to use tools.

Provisioning the SQL warehouse

Before creating a Genie agent, we must provision a SQL warehouse, as Genie relies on a warehouse to query tables. We will create this warehouse using the Databricks SDK. In this example, we deliberately choose

the smallest serverless warehouse size, configure auto-termination after five minutes of inactivity, and apply a usage policy so that costs can be tracked and allocated later.

While we used DABs to create a warehouse in [Chapter 6](#), we are using the SDK here for a specific reason: at the time of writing this book, creating a Genie agent was not yet supported via DABs. To keep our provisioning logic consistent within this workflow, we'll manage both the warehouse and the Genie agent through the SDK:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service import sql
from databricks.sdk.service.sql import CreateWarehouseRequestWarehouseType

w = WorkspaceClient()

created = w.warehouses.create(
    name="2XS_arxiv_warehouse",
    cluster_size="2X-Small",
    max_num_clusters=1,
    auto_stop_mins=5,
    warehouse_type=CreateWarehouseRequestWarehouseType("PRO"),
    enable_serverless_compute=True,
    tags=sql.EndpointTags(
        custom_tags=[sql.EndpointTagPair(key="project",
                                         value="arxiv_curator")]
    ),
).result()

warehouse_id=created.id
```

Creating the Genie agent

We can create a Genie agent via the UI, API, or Databricks SDK. Below is an example of how to create a Genie agent using SDK, with access to only one table (`arxiv_papers`) under the defined catalog and schema:

```
import json

from arxiv_curator.config import ProjectConfig

cfg = ProjectConfig.from_yaml("../project_config.yml")
catalog = cfg.catalog
schema = cfg.schema
```

```

serialized_space = {
    "version": 1,
    "data_sources": {
        "tables": [
            {
                "identifier": f"{catalog}.{schema}.arxiv_papers",
                "column_configs": [
                    {"column_name": "authors"},
                    {"column_name": "ingest_ts", "get_example_values": True},
                    {"column_name": "paper_id", "get_example_values": True},
                    {
                        "column_name": "pdf_url",
                        "get_example_values": True,
                        "build_value_dictionary": True,
                    },
                    {"column_name": "processed", "get_example_values": True},
                    {"column_name": "published", "get_example_values": True},
                    {
                        "column_name": "summary",
                        "get_example_values": True,
                        "build_value_dictionary": True,
                    },
                    {
                        "column_name": "title",
                        "get_example_values": True,
                        "build_value_dictionary": True,
                    },
                    {
                        "column_name": "volume_path",
                        "get_example_values": True,
                        "build_value_dictionary": True,
                    },
                ],
            }
        ]
    },
}

space = w.genie.create_space(
    warehouse_id=warehouse_id,
    serialized_space=json.dumps(serialized_space),
    title="arxiv-curator-space",
)

```

We can retrieve the serialized space definition for an existing Genie agent by using the `get_space()` function:

```
space_id = space.space_id
```

```
space = w.genie.get_space(space_id=space_id, include_serialized_space=True)
json.loads(space.serialized_space)
```

Interacting with Genie

We can interact with the Genie agent by using SDK. For example, we can start a conversation and ask it to find the last 10 published papers that are available in the metadata table:

```
conversation = w.genie.start_conversation_and_wait(
    space_id=space.space_id,
    content="Find the last 10 papers published")
```

We can then inspect what the returned object looks like (see [Figure 7-8](#)) and notice that Genie translated the original question to a SQL query.

```
conversation.as_dict()
{'attachments': [{'attachment_id': '01f0e97df71e1c3b869aa1019b1b55fb',
  'query': {'description': 'You want to see the 10 most recently published papers, showing their ID, title, publication timestamp, and PDF link.',
    'query': 'SELECT `paper_id`, `title`, `published`, `pdf_url`\nFROM `mlops_dev`.`arxiv`.`arxiv_papers`\nWHERE `published` IS NOT NULL\nORDER BY `published` DESC\nLIMIT 10',
    'query_result_metadata': {'row_count': 10},
    'statement_id': '01f0e97d-f72b-1afb-94c6-917f2e7b5a92'}]},
  {'attachment_id': '01f0e97df85011c0ae7b3b6cf8c3d7c7',
    'suggested_questions': {'questions': ['What are the titles and publication dates of the last 10 papers published?',
      'Who are the authors of the last 10 papers published?',
      'What are the summaries of the last 10 papers published?']}]},
  'content': 'Find the last 10 papers published',
  'conversation_id': '01f0e97df52c113485f74e8624c60bd2',
  'created_timestamp': 1767538764299,
  'id': '01f0e97df53b1a72887481c15029acd3',
  'last_updated_timestamp': 1767538770147,
  'message_id': '01f0e97df53b1a72887481c15029acd3',
  'query_result': {'row_count': 10},
  'statement_id': '01f0e97d-f72b-1afb-94c6-917f2e7b5a92',
  'space_id': '01f0c3f882e41bb9be96b4ddf295d2a4',
```

Figure 7-8. Genie conversation output

Now, let's ask a question within the same conversation (Genie keeps the message history within the conversation) and request to return the list of authors of the last 10 publications:

```
message = w.genie.create_message_and_wait(
    space_id=space.space_id,
    conversation_id=conversation.conversation_id,
    content="Return the list of authors of the last 10 papers published")
```

We'll see in the output that Genie was able to successfully answer the question (see [Figure 7-9](#)).

```

message.as_dict({
  'attachments': [{'attachment_id': '01f0e97e33b71d68afab1c096dae2855',
    'text': {'content': 'Here are the authors of the last 10 papers published:\n\n1. 2511.14761v1: Keya Hu, Ali Cy, Linlu Qiu, Xiaoman Delores Ding, Runqian Wang, Yeyin Eva Zhu, Jacob Andreas, Kaiming He\n2. 2511.14730v1: Parya Dolatyabi, Mahdi Khodayar\n3. 2511.14728v1: Zoltán Kovács, Xicheng Peng\n4. 2511.14719v1: Yifan Wang, Liya Ji, Zhanhan Ke, Harry Yang, Ser-Nam Lim, Qifeng Chen\n5. 2511.14715v1: Abolfazl Younesi, Leon Kiss, Zahra Najafabadi Samani, Juan Aznar Poveda, Thomas Fahringer\n6. 2511.14702v1: Farheen Ramzan, Yusuf Kiberu, Nimesh Jathanna, Meryem Jabrane, Vicente Grau, Shahnaz Jamil-Copley, Richard H. Clayton, Chen, Chen\n7. 2511.14694v1: Rui Zhu, Xiaopu Zhou, Haixu Tang, Stephen W. Scherer, Lucila Ohno-Machado\n8. 2511.14691v1: Kallol Mondal, Ankush Kumar\n9. 2511.14689v1: Shiya r Jamo\n10. 2511.14688v1: Clovis Gladstone, Zhao Fang, Spencer Dean Stewart'}}],
  'attachment_id': '01f0e97e34741a6f85b3b4e099ed0077',
  'suggested_questions': {'questions': ['What are the titles of the last 10 papers published?',
    'What are the publication dates of the last 10 papers published?',
    'How many authors contributed to each of the last 10 papers published?']}},
  'content': 'Return the list of authors of the last 10 papers published',
  'conversation_id': '01f0e97df52c113485f74e8624c60bd2',
  'created_timestamp': 1767538865847,
  'id': '01f0e97e31c219d4807d0f2766b7230a',
  'last_updated_timestamp': 1767538870410,
  'message_id': '01f0e97e31c219d4807d0f2766b7230a',
  'space_id': '01f0c3f882e41bb9be96b4ddf295d2a4',
  'status': 'COMPLETED',

```

Figure 7-9. Genie message output

Having established our AI Search index for semantic retrieval and our Genie agent for analytical queries, we now have the two foundational *sources of context* for our agent.

The next challenge is exposing these capabilities to the agent in a standardized, scalable way. To do this, we will utilize Databricks-managed MCP servers. This approach allows the agent to interact with both structured and unstructured data through a unified protocol, which we will explore in the following section.

MCP Servers on Databricks and Tool Calling

The Model Context Protocol (MCP) is an open standard that was released by Anthropic in November 2024 to solve a fundamental problem: how to allow AI agents access to the data and tools they need in a standardized way without requiring custom integrations every time.¹⁶

Since MCP's release, its adoption has grown fast, and MCP has become a standard for AI development. Almost any tool you can think of has MCP integration today: [Hugging Face](#), [GitHub](#), [Postman](#), and many more.

Managed MCP ecosystems on Databricks

You may have noticed that both the vector search index and the Genie agent have appeared under MCP servers (see [Figure 7-10](#)) without any configuration from our side, alongside the Unity Catalog functions we created in [Chapter 4](#). That's because all these functionalities are treated as Databricks-managed MCP by default.

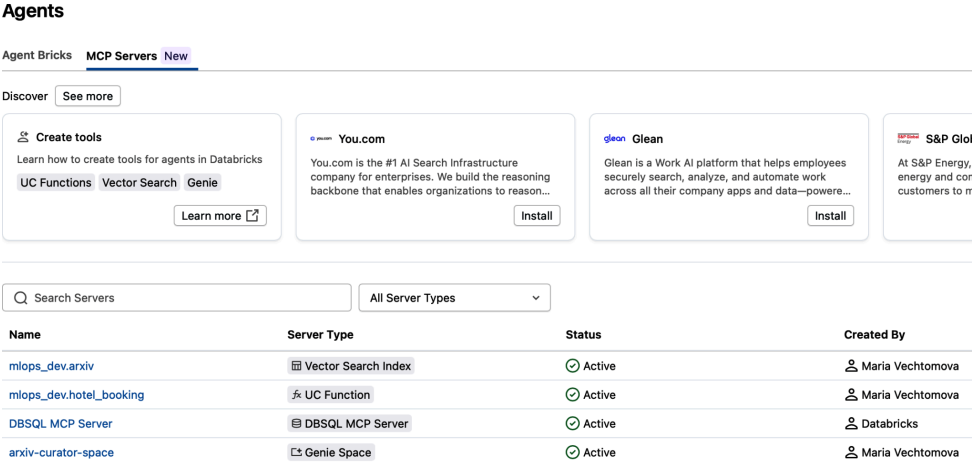


Figure 7-10. Managed MCP servers on Databricks

Table 7-1 outlines the available managed servers.

Table 7-1. Overview of Databricks-managed MCP servers

MCP server	URL pattern	Description	Available tools and parameters
AI Search index	<code>https://{host}/api/2.0/mc p/vector-search/{catalog}/{s chema}</code>	This queries an AI Search index to find relevant chunks.	<code>{catalog}__{sche ma}__{index}: que ry(str)</code>
Genie agent	<code>https://{host}/api/2.0/mc p/genie/{genie _space_id}</code>	This queries Genie agent (formerly, Genie space).	<code>query_space_{spa ce_id}: query (st r), conversation_ id (str); poll_response_{s pace_id}: convers ation_id (str); m essage_id (str)</code>
Unity Catalog functions	<code>https://{host}/api/2.0/mc p/functions/{c atalog}/{schem a}</code>	This invokes a unity catalog function.	<code>{catalog}__{sche ma}__{fn}; parameters are defined by users when functions get created.</code>
DBSQL	<code>https://{host}/api/2.0/mc p/sql</code>	This runs AI-generated SQL.	<code>execute_sql: que ry(str); execute_sql_read _only: query(st r); poll_sql_result: statement_id (st r)</code>

One of the known limitations for AI Search index is that only indexes with Databricks managed embeddings are supported. Extra parameters such as query type, the reranker, and filters are not supported out of the box.

We can also set up an external MCP server on Databricks via Databricks Marketplace or HTTP connection, create a governance layer on top, and have it handle authentication and access.¹⁷

Implementing tool calling

To use the MCP servers on Databricks, we first need a consistent way to represent a tool to the agent. The following `ToolInfo` model holds the three things the agent needs for each tool: its `name`, the `spec` that describes the tool to the model, and the `exec_fn` that runs it when the model decides to call it:

```
from collections.abc import Callable
from pydantic import BaseModel

class ToolInfo(BaseModel):
    name: str
    spec: dict
    exec_fn: Callable
```

We can call tools we define in Databricks MCPs by using the `DatabricksMCPClient`. To initialize it, we need to provide a server URL and a `WorkspaceClient`. Here, we create a `create_managed_exec_fn()` function, which is a wrapper around tool calling:

```
from databricks.sdk import WorkspaceClient
from databricks_mcp import DatabricksMCPClient

def create_managed_exec_fn(
    server_url: str, tool_name: str, w: WorkspaceClient
) -> Callable:
    def exec_fn(**kwargs):
        client = DatabricksMCPClient(server_url=server_url, workspace_client=w)
        response = client.call_tool(tool_name, kwargs)
        return "".join([c.text for c in response.content])
    return exec_fn
```

Finally, we implement the `create_mcp_tools()` function, which takes a list of MCP server URLs and a `WorkspaceClient` as arguments and returns the list of `ToolInfo` objects we defined earlier. We must declare the function as `async` because `DatabricksMCPClient` exposes an asynchronous API. MCP clients are asynchronous by design because the problems they solve—like tool use, context gathering, and model coordination—are inherently I/O bound, multistep, and latency sensitive. Asynchronicity isn't an implementation detail; it's what makes MCP practical at scale:

```

async def create_mcp_tools(w: WorkspaceClient, url_list: list[str]) -> list[Tool]:
    tools = []
    for server_url in url_list:
        mcp_client = DatabricksMCPClient(server_url=server_url, workspace_client=w)
        mcp_tools = mcp_client.list_tools()
        for mcp_tool in mcp_tools:
            input_schema = mcp_tool.inputSchema.copy() if mcp_tool.inputSchema else {}
            tool_spec = {
                "type": "function",
                "function": {
                    "name": mcp_tool.name,
                    "parameters": input_schema,
                    "description": mcp_tool.description or f"Tool: {mcp_tool.name}"
                }
            }
            exec_fn = create_managed_exec_fn(server_url, mcp_tool.name, w)
            tools.append(ToolInfo(name=mcp_tool.name, spec=tool_spec, exec_fn=exec_fn))
    return tools

```

This code is defined under the `mcp` module of the `arxiv_curator` package. We'll use the `create_mcp_tools()` function later in [Chapter 8](#).

We've covered how agents access external data and how MCP servers expose tools in a standardized way. But there's a third piece: maintaining state across turns within a session.

Agent Memory with Lakebase

When you deploy an agent behind a serving endpoint, you decouple the agent from its caller. Each request arrives independently, and the agent has no built-in way to know this is turn five of an ongoing conversation. Yet the user expects continuity. They ask about their recent order and then follow up with “Can you change the shipping address?” The agent needs to know which order they mean.

Session persistence solves this problem. The caller passes a session identifier with each request, and the agent uses it to retrieve and update the conversation state. This keeps the agent stateless at the infrastructure level while maintaining conversational context where it matters.

This is where Lakebase comes in. We already used it in [Chapter 4](#) for Feature Serving and Model Serving with feature lookup, but it's not limited to ML workflows. Lakebase is a fully managed Postgres database, so it's a natural fit for storing agent session state.

The core pattern is straightforward: you store messages and retrieve them by session ID. From there, you can evolve the design. For example, instead of replaying the full conversation every time, you can introduce a *summarization step* that periodically compresses earlier turns into a shorter representation. This reduces token usage, improves latency, and keeps the agent focused on the most relevant context while still preserving continuity.

For simplicity's sake, we'll leave summarization out of scope for this book. Instead, let's focus on the basic pattern: storing messages and retrieving them by session ID.

Lakebase Autoscaling for agent workloads

In [Chapter 4](#), we introduced Lakebase for Feature Serving and created a Lakebase project using the Feature Engineering client. However, we did not explore [Lakebase Autoscaling's capabilities](#) in detail.

Its key features include the following:

Autoscaling

Compute resources scale dynamically with demand, so you don't need to provision for peak traffic in advance.

Scaling to zero

Inactive compute scales to 0 after timeout to minimize costs.

Branches

These allow you to create isolated database branches for development, testing, and experimentation.

Read replicas

You can use dedicated read-only replicas for read queries. By separating reads from writes, you reduce load on the primary database and improve overall throughput and latency.

Instant restore

You can create a new branch from any point in time within your restore window, so it's easier to recover data or debug issues.

Lakebase Autoscaling project structure

Understanding the Lakebase Autoscaling project structure helps you organize and manage your resources effectively. A *project* is the top-level organizational unit in Lakebase Autoscaling, and it acts as the primary container for all your resources. If you've worked with Lakebase Provisioned before, you can think of a project as playing a role similar to that of an instance.

A project groups together your databases, branches, computes, and related configuration. It also defines defaults such as compute settings, restore windows, and update policies that apply across all branches within the project.

At the top level, a project contains one or more branches. Each branch is isolated and includes its own compute, roles, and databases. This structure allows you to manage environments cleanly, experiment without impacting production, and scale workloads independently within a single project:

```
Project
├── Branches (main, development, staging, etc.)
│   ├── Computes (endpoints)
│   ├── Roles (Postgres roles)
│   └── Databases (Postgres databases)
```

Creating a Lakebase Autoscaling project

We'll start by creating a Lakebase Autoscaling project programmatically, using the Postgres API, which is part of the Databricks SDK. This approach allows you to define your IaC and integrate project creation directly into your workflows.

In this example, we configure a new project with autoscaling settings, including minimum and maximum compute capacity and an automatic suspend timeout to enable scale-to-zero behavior. This ensures the system can scale with demand while avoiding unnecessary costs when idle. Here, we also associate the project with a usage policy for cost tracking:

```
import json
import urllib.parse
from uuid import uuid4

import psycopyg
```

```

from databricks.sdk import WorkspaceClient
from databricks.sdk.service.postgres import (
    PostgresAPI,
    Project,
    ProjectDefaultEndpointSettings,
    ProjectSpec,
)
from google.protobuf.duration_pb2 import Duration
from loguru import logger

from arxiv_curator.config import ProjectConfig

cfg = ProjectConfig.from_yaml("../project_config.yml")

w = WorkspaceClient()
pg_api = PostgresAPI(w.api_client)

project_id = "arxiv-agent-lakebase"

try:
    project = pg_api.get_project(name=f"projects/{project_id}")
except Exception:
    project = pg_api.create_project(
        project_id=project_id,
        project=Project(
            spec=ProjectSpec(
                display_name=project_id,
                usage_policy_id=cfg.usage_policy_id,
                default_endpoint_settings=ProjectDefaultEndpointSettings(
                    autoscaling_limit_min_cu=1,
                    autoscaling_limit_max_cu=4,
                    suspend_timeout_duration=Duration(seconds=300),
                ),
            ),
        ),
    ).wait()

```

Now that we have set up the Lakebase project, let's create a connection, define a table, and try to insert and read data.

Interacting with Lakebase

To create a connection, we need a host. Due to Lakebase's multilevel structure, we first retrieve the branch (which is, by default, the production branch). Each branch has an associated compute endpoint, which provides the host required to establish the connection:

```

default_branch = next(iter(pg_api.list_branches(parent=project.name)))
endpoint = next(iter(pg_api.list_endpoints(parent=default_branch.name)))
host = endpoint.status.hosts.host

```

We then can use the Postgres API again to generate the database credentials and construct the connection string:

```

user = w.current_user.me()
pg_credential = pg_api.generate_database_credential(endpoint=endpoint.name)
username = urllib.parse.quote_plus(user.user_name)
conn_string = (
    f"postgresql://{username}:{pg_credential.token}@{host}:5432/"
    "databricks_postgres?sslmode=require"
)

```

Because Lakebase is a Postgres database, we can work with it using standard Python libraries. We use [psycopg3](#), the modern PostgreSQL adapter for Python, to open a connection, and then, we use that connection to create a table for storing session-based conversation messages:

```

import psycopg

with psycopg.connect(conn_string) as conn:
    # Create session_messages table
    conn.execute("""
        CREATE TABLE IF NOT EXISTS session_messages (
            id SERIAL PRIMARY KEY,
            session_id TEXT NOT NULL,
            message_data JSONB NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_session_messages_session_id
        ON session_messages(session_id)
    """)

```

After we create the table, we can insert messages into it:

```

test_session_id = f"test-session-{uuid4()}"
test_messages = [
    {"role": "user", "content": "Hello, what can you help me with?"},
    {"role": "assistant", "content": "I can help you find research papers."},
    {"role": "user", "content": "Find papers about LLM reasoning"},
]

```

```

with psycopg.connect(conn_string) as conn:
    for msg in test_messages:
        conn.execute(
            "INSERT INTO session_messages (session_id, message_data) "
            "VALUES (%s, %s)",
            (test_session_id, json.dumps(msg)),
        )

```

And then, we can successfully read them back:

```

with psycopg.connect(conn_string) as conn:
    result = conn.execute(
        """
        SELECT message_data, created_at FROM session_messages
        WHERE session_id = %s
        ORDER BY created_at ASC
        """,
        (test_session_id,),
    ).fetchall()

    logger.info(f"Loaded {len(result)} messages:")
    for row in result:
        logger.info(f"    [{row[1]]} {row[0]}")

```

The agent can use this mechanism to load the messages from Lakebase when it receives a request and store messages before it generates output.

So far, we have worked with a single database connection. That is fine for demonstration purposes, but it does not scale. In a real application and especially in an agent system that handles concurrent requests, opening and closing a connection for every request adds unnecessary overhead. We solve this with a connection pool.

Utilizing a connection pool

A *connection pool* maintains a set of reusable database connections. Instead of opening a new connection each time, requests borrow a connection from the pool and return it when done. This reduces latency, improves throughput, and allows the system to handle multiple requests more efficiently.

In the memory module of our custom package, we define a `LakebaseMemory` class that uses a connection pool to interact with

Lakebase and manage session persistence. Let's walk through the implementation step by step.

The class takes a host and instance name during initialization, but it doesn't immediately establish connections. Instead, the connection pool is created lazily, only when it's first needed:

```
import json
import os
import urllib.parse
from typing import Any

import psycopg
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.postgres import PostgresAPI
from loguru import logger
from psycopg_pool import ConnectionPool

class LakebaseMemory:
    """Handles session message persistence using Lakebase (PostgreSQL)."""

    def __init__(
        self,
        project_id: str,
    ) -> None:
        self.project_id = project_id
        self._pool: ConnectionPool | None = None
```

Connecting to Lakebase requires exchanging Databricks credentials for a short-lived database token. This class supports two authentication modes: a *service principal* for production use and *user credentials* for local development and testing.

In production, we must add the service principal to the Lakebase project and grant it the necessary permissions to read from and write to the database:

```
def _get_connection_string(self) -> str:
    """Build connection string for Lakebase.

    Supports two authentication modes:
    - SPN (production): Needs LAKEBASE_SP_CLIENT_ID,
      LAKEBASE_SP_CLIENT_SECRET, LAKEBASE_SP_HOST
    - User (local testing): Uses default WorkspaceClient auth
    """
```

```

# Use dedicated Lakebase SPN env vars to avoid overriding
# the default WorkspaceClient auth used by MCP tools
client_id = os.environ.get("LAKEBASE_SP_CLIENT_ID")
client_secret = os.environ.get("LAKEBASE_SP_CLIENT_SECRET")
host = os.environ.get("LAKEBASE_SP_HOST")

if client_id and client_secret and host:
    w = WorkspaceClient(
        host=host,
        client_id=client_id,
        client_secret=client_secret,
    )
else:
    w = WorkspaceClient()

pg_api = PostgresAPI(w.api_client)

```

Notice that our implementation uses *Lakebase-specific environment variables* (`LAKEBASE_SP_CLIENT_ID`, `LAKEBASE_SP_CLIENT_SECRET`, and `LAKEBASE_SP_HOST`) instead of the standard Databricks variables (`DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID`, and `DATABRICKS_CLIENT_SECRET`). This avoids conflicts with other components, particularly MCP resources that rely on the default authentication chain. We'll revisit this distinction later, when logging and registering the agent.

The connection pool is created on the first use with the `_get_pool()` function. Since database tokens are short lived, connections can fail when credentials expire. The `_reset_pool()` method handles this. If we hit an `OperationalError`, we close the pool so the next request gets fresh tokens:

```

def _get_pool(self) -> ConnectionPool:
    """Get or create connection pool."""
    if self._pool is None:
        conn_string = self._get_connection_string()
        self._pool = ConnectionPool(conninfo=conn_string, min_size=1, max_size=5)
    return self._pool

def _reset_pool(self) -> None:
    """Reset pool to force new credentials on next use."""
    if self._pool is not None:
        self._pool.close()
        self._pool = None

```

The core functionality is straightforward: we load and save messages by session ID. When loading, we retrieve all messages for a session ordered by timestamp:

```
def load_messages(self, session_id: str) -> list[dict[str, Any]]:
    """Load previous messages for a session."""
    try:
        with self._get_pool().connection() as conn:
            self._ensure_messages_table(conn)
            result = conn.execute(
                """
                SELECT message_data FROM session_messages
                WHERE session_id = %s
                ORDER BY created_at ASC
                """,
                (session_id,),
            ).fetchall()
            return [row[0] for row in result]
    except psycopg.OperationalError:
        self._reset_pool()
        raise
    except Exception as e:
        logger.warning(f"Failed to load session messages: {e}")
        return []
```

When saving, we append new messages to the session:

```
def save_messages(
    self, session_id: str, messages: list[dict[str, Any]]
) -> None:
    """Append messages to a session."""
    try:
        with self._get_pool().connection() as conn:
            self._ensure_messages_table(conn)
            for msg in messages:
                conn.execute(
                    "INSERT INTO session_messages (session_id, message_data) "
                    "VALUES (%s, %s)",
                    (session_id, json.dumps(msg)),
                )
    except psycopg.OperationalError:
        self._reset_pool()
        raise
    except Exception as e:
        logger.warning(f"Failed to save session messages: {e}")
```

We'll use the `LakebaseMemory` class when defining the agent in [Chapter 8](#).

Conclusion

In this chapter, we built up the context an agent needs, step by step. We began with the different ways to use foundation models on Databricks. We then processed raw arXiv PDFs into parsed, chunked text and indexed it in Databricks AI Search so it could be retrieved, and we set up a Genie agent for natural-language queries over structured data. Finally, we saw how both Databricks AI Search and Genie can be exposed to an agent as tools through MCP.

So far, we have assembled the context and tools an agent can draw on, but not the agent itself. We have not yet defined its logic: how it decides which tool to call, when to retrieve, and how it sequences those steps into a response. That logic is the subject of the next chapter, along with two pieces that make the agent operational: MLflow flavors for agents, which give us a standard way to package and serve it, and MLflow tracing, which lets us see what the agent did on each request.

- ¹ You can find the definition in the [OpenAI SDK guides](#).
- ² See [the Databricks documentation on AI functions](#).
- ³ Consult your cloud-specific documentation for more details. For example, see [the AWS documentation](#).
- ⁴ You can find supported models in [the documentation](#).
- ⁵ For further details, consult [the community discussion](#).
- ⁶ For further details, consult [the documentation on benchmarking](#).
- ⁷ For further details, consult [the documentation on networking](#).
- ⁸ Databricks solution architects Sara Dooley and Austin Zaccor wrote [a blog that explains how to achieve the Custom Provider solution](#).
- ⁹ Notice that we refer to model families rather than specific versions. Individual releases change quickly, and many are retired within months, so for the current options on Databricks, consult [the foundation model overview documentation](#).

- 10** Note that we use the `time.sleep(3)` command to suspend the execution for three seconds to avoid reaching API limits. For more information, see <https://oreil.ly/Na6hZ>.
- 11** For details on AI Search, consult [the Databricks documentation](#).
- 12** Consult [Vector Search Python SDK documentation](#) for exact details.
- 13** For more details, consult [the documentation on querying the AI Search index](#).
- 14** This [blog by Redis](#) explains how this algorithm can improve search.
- 15** For more information, consult [the Databricks documentation](#).
- 16** Anthropic, “Introducing the Model Context Protocol,” November 25, 2024, <https://oreil.ly/Hh0UK>.
- 17** For more information, consult [the Databricks documentation on external MCPs](#).

Previous chapter

< [6. Monitoring ML Applications](#)

Next chapter

[8. MLflow for GenAI](#) >